Rapport de TD : Visual Studio et débogage

Enseignement .NET · Génie Informatique · École Supérieure Polytechnique (UCAD)
Chargé de cours : E. H. Ousmane Diallo · Année 2026/2027
Étudiant : Rassoul · Rendu du 4 novembre 2026

Résumé

Ce rapport présente la prise en main et le débogage de l'application Boutique Diayma, une application ASP.NET Core MVC en .NET Core 2.0.

- **Projet :** la solution contient un seul projet, `Diayma`.
- **Exécution :** l'application a été compilée avec 0 erreur et testée sous Linux.
- **Bugs :** 2 bugs principaux et 4 autres anomalies ont été reproduits par des requêtes réelles.
- **Parcours :** 30 étapes ont été relevées entre le démarrage et l'affichage des produits.

Environnement et limites

Élément	Valeur
Système	Linux, avec un conteneur Debian 9
SDK	.NET Core 2.1.818
Runtime utilisé	2.1.30 (le projet cible `netcoreapp2.0`)
Éditeur	VS Code ; Visual Studio n'existe pas sous Linux

- Le parcours de la question 8 a été relevé par des traces, pas pas à pas dans un débogueur.
- L'exécutable Windows a été généré sous Linux et n'a pas pu être lancé sous Windows depuis ce poste.
- Le symbole `¤` affiché devant les prix sous Linux n'est pas compté comme un bug, faute de vérification sous Windows.

Dépôt d'origine : https://github.com/devehod/BoutiqueDiayma2025

1. Récupération et exécution

Le code a été récupéré avec `git clone`, puis compilé et exécuté sous Linux, dans un conteneur contenant le SDK .NET Core 2.1.818. La compilation donne 0 erreur et 0 avertissement.

Visual Studio n'existant pas sous Linux, la fenêtre d'avertissement sur les packages vulnérables n'a pas été observée.

2. Projets de la solution

La solution `Diayma.sln` contient un seul projet : `Diayma` (`P2FixAnAppDotNetCode/Diayma.csproj`).

3. Version du SDK

- Le projet cible `netcoreapp2.0`, soit .NET Core 2.0, qui n'est plus pris en charge.

- Il utilise le package `Microsoft.AspNetCore.All` en version 2.0.6.
- D'après les notes de version officielles, le runtime 2.0.6 correspond au SDK 2.1.101.

4. Installation du SDK

SDK utilisé : 2.1.818, avec l'image officielle `mcr.microsoft.com/dotnet/core/sdk:2.1`. Ce SDK compile le projet en `netcoreapp2.0`. Cette image contient le runtime 2.1.30 : l'application s'y exécute sans le runtime 2.0.

```
dotnet build Diayma.sln
cd P2FixAnAppDotNetCode
dotnet run
```

5. Dépôt personnel

https://github.com/rassoul7/BoutiqueDiayma2025

6. Bugs trouvés

Bug 1 : le total du panier ignore les quantités

Reproduction : ajouter deux fois le XIAOMI Redmi 13C, à 88 000, puis ouvrir le panier.

Valeur	Affiché	Attendu
Quantité	2	2
Sous-total	176 000	176 000
Total	**88 000**	176 000

Cause : `Models/Cart.cs`, ligne 66 : `Sum(x => x.Product.Price)` additionne les prix sans les multiplier par la quantité. La moyenne (ligne 77) ignore aussi les quantités.

Bug 2 : le stock ne diminue jamais après une commande

Reproduction : commander 2 XIAOMI, dont le stock est de 10, puis revenir à l'accueil. Le stock affiché reste à **10** au lieu de 8.

Cause : `Startup.cs` enregistre `ProductRepository` en `AddTransient`, et son constructeur vide puis recrée la liste statique `_products` (`ProductRepository.cs`, lignes 15 et 16). La liste est donc réinitialisée à chaque requête.

Autres anomalies constatées

- **Panier partagé :** `ICart` est un singleton. Un second navigateur, sans lien avec le premier, voit le même panier.
- **Vente au-delà du stock :** 11 Galaxy A25 ont pu être commandés alors que le stock est de 10.
- **Messages d'erreur non traduits :** une commande vide affiche `ErrorMissingName` ou `CartEmpty` au lieu du texte, y compris en français.
- **Espagnol sans effet :** cette langue est proposée, mais l'interface reste en anglais. Il n'existe ni culture `es` déclarée ni fichier de traduction.

7. Points d'arrêt

Emplacement	Instruction	Atteint
`Startup`, ligne 20	`Configuration = configuration;`	Au démarrage, une seule fois
`ProductController`, ligne 15	`_productService = productService;`	À l'affichage de l'accueil
`CartSummaryViewComponent`, ligne 12	`_cart = cart as Cart;`	À l'affichage du layout, donc à chaque page
`CartController`, ligne 15	`_cart = pCart;`	Seulement en allant sur le panier
`OrderController`, ligne 17	`_cart = pCart;`	Seulement en allant sur la commande

Sur l'écran d'accueil, les points d'arrêt de `CartController` et `OrderController` ne sont donc pas atteints.

8. Parcours avant l'affichage des produits

Méthode : l'ordre ci-dessous est un relevé réel, obtenu sur une copie temporaire du code avec une trace au début de chaque méthode, pendant le chargement de `/`. Il n'a pas été obtenu pas à pas dans Visual Studio. Les modes indiqués sont ceux à utiliser pour rejouer ce parcours au débogueur.

Démarrage de l'application

#	Namespace	Classe	Méthode	Mode conseillé
1	`P2FixAnAppDotNetCode`	`Program`	`Main`	Détaillé (F11)
2	`P2FixAnAppDotNetCode`	`Program`	`BuildWebHost`	Détaillé (F11)
3	`P2FixAnAppDotNetCode`	`Startup`	constructeur	Point d'arrêt ligne 20
4	`P2FixAnAppDotNetCode`	`Startup`	`ConfigureServices`	Principal (F10)
5	`P2FixAnAppDotNetCode`	`Startup`	`Configure`	Principal (F10)

Requête `GET /`

La route par défaut `{controller=Product}/{action=Index}/{id?}` mène à `ProductController.Index`.

#	Namespace	Classe	Méthode	Mode conseillé
6	`P2FixAnAppDotNetCode.Models.Repositories`	`ProductRepository`	constructeur	Détaillé (F11)
7	`P2FixAnAppDotNetCode.Models.Repositories`	`ProductRepository`	`GenerateProductData`	Détaillé (F11)
8 à 16	`P2FixAnAppDotNetCode.Models`	`Product`	constructeur, 9 fois	Principal (F10) sur chaque ligne
17	`P2FixAnAppDotNetCode.Models.Repositories`	`OrderRepository`	constructeur	Sortant (Maj+F11)
18	`P2FixAnAppDotNetCode.Models.Services`	`ProductService`	constructeur	Sortant (Maj+F11)

#	Namespace	Classe	Méthode	Mode conseillé
19	`P2FixAnAppDotNetCode.Controllers`	`ProductController`	constructeur	Point d'arrêt ligne 15
20	`P2FixAnAppDotNetCode.Controllers`	`ProductController`	`Index`	Détaillé (F11)
21	`P2FixAnAppDotNetCode.Models.Services`	`ProductService`	`GetAllProducts`	Détaillé (F11)
22	`P2FixAnAppDotNetCode.Models.Repositories`	`ProductRepository`	`GetAllProducts`	Sortant (Maj+F11)
23 à 25	Vues Razor	`_ViewStart`, `Product/Index`, `_Layout`	rendu	Principal (F10)
26	`P2FixAnAppDotNetCode.Components`	`CartSummaryViewComponent`	constructeur	Point d'arrêt ligne 12
27	`P2FixAnAppDotNetCode.Components`	`CartSummaryViewComponent`	`Invoke`	Principal (F10)
28	Vue Razor	`CartSummary/Default`	rendu	Principal (F10)
29	`P2FixAnAppDotNetCode.Components`	`LanguageSelectorViewComponent`	`Invoke`	Principal (F10)
30	Vue Razor	`LanguageSelector/Default`	rendu	Principal (F10)

`Cart` et `LanguageService` sont aussi créés par injection de dépendances, mais ils n'ont pas de constructeur écrit dans le code : ils n'apparaissent donc pas dans le relevé.

Choix des modes :

- **Détaillé (F11)** pour entrer dans une méthode de l'application dont on veut voir le contenu.
- **Principal (F10)** pour passer une ligne sans entrer dedans : configuration du framework, création répétée des produits, rendu.
- **Sortant (Maj+F11)** pour terminer la méthode en cours et revenir à l'appelant.

9 et 10. Exécutable Windows

```
cd P2FixAnAppDotNetCode
dotnet publish Diayma.csproj -c Release -r win-x64 --self-contained -o publication-win-x64
```

Lien de téléchargement : indiqué dans le README du dépôt.

11. Option : langue wolof

La langue wolof a été ajoutée à l'interface :

- Le sélecteur propose « Wolof » et un fichier `.wo.resx` existe pour chaque écran.
- Les textes passent en wolof, tandis que les formats de nombres, de dates et de monnaie restent ceux du français, comme demandé (`RequestCulture("fr", "wo")`).

- Le SDK en ligne de commande ne reconnaît pas la culture `wo`. Deux cibles ajoutées à `Diayma.csproj` génèrent quand même le dossier `wo`.

Contrôle réalisé : en wolof, l'accueil, le panier et la commande s'affichent dans cette langue. En français, les textes restent corrects.

Limite : ces traductions n'ont pas été relues par un locuteur natif.